

Guessing-Game – Projektkonzept

Idee

Eine interaktive Web-Spiel-Website mit einem einzigartigen Guessing-Game-Konzept. Ziel: Geld verdienen, wie es Trendspiele/Apps à la "Imposter" vorgemacht haben – kein reiner Klon bestehender Wordle-artiger Spiele.

Thema: Minecraft-Fakten

Fußballer-Rate-Spiele gibt's schon zu viele (Footdle, Statdle, ...). Stattdessen: **obskure, wenig bekannte Minecraft-Fakten und Werte.**

Beispiele:

- Wie viele Blocks kann ein Redstone-Signal weitergeben?
- Wie schnell sind Minecarts?
- Bei wie vielen Herzen poppt ein Totem der Unsterblichkeit? (0,5 Herzen)
- Welcher Mob gibt dir unendlich Essen? (Mooshroom)

Content-Kategorien: Redstone, Mobs, Crafting, Weltgenerierung, Versionsgeschichte.

Vorteil: nahezu unerschöpfliche Faktenbasis über das Minecraft-Wiki, große treue Zielgruppe mit hoher Teilbereitschaft (viraler Faktor).

Zwei Spielmodi

Solo-Modus

- Tägliches Rätsel (wie Wordle): ein Fakt pro Tag, alle Spieler raten denselben
- Mehrere Versuche mit Feedback ("zu hoch"/"zu niedrig"/Nähe zur Lösung)
- Nach Auflösung: kurzer Fun Fact zur Erklärung
- Highscore-/Streak-Zähler zur Bindung

Multiplayer-Modus (Alleinstellungsmerkmal)

- Raum erstellen, Code/Link mit Freunden teilen
- **Buzzer-System:** Frage wird an alle gleichzeitig gezeigt
 - Wer zuerst buzzert, bekommt **10 Sekunden** Zeit für eine Antwort
 - Bei falscher Antwort oder Timeout: Buzzer öffnet sich wieder für die verbleibenden Mitspieler
 - Punkte nach Richtigkeit (und ggf. Nähe bei Zahlenfragen)

- Erfordert eine serverseitige Buzzer-Logik (Echtzeit, fairer Zeitstempel), technisch also WebSockets nötig (z. B. Node.js + Socket.io oder Firebase Realtime Database)

Zwei Fragetypen (Datenmodell)

Typ „number“ – für Werte mit Einheit:

```
{
  type: "number",
  question: "Wie viele Blocks kann ein Redstone-Signal weitergeben?",
  answer: 15,
  unit: "Blocks",
  tolerance: 0,          // z. B. bei Minecart-Speed ggf. ± Toleranz
  category: "Redstone",
  funFact: "..."
```

Antworteingabe: Zahl eintippen, Server prüft exakten Wert oder Toleranzbereich.

Typ „entity“ – für Mob-/Item-/Block-Namen:

```
{
  type: "entity",
  question: "Welcher Mob kann dir unendlich Essen geben?",
  answer: "Mooshroom",
  acceptedAnswers: ["Mooshroom", "Pilzkuh"],
  category: "Mobs",
  funFact: "..."
```

Antworteingabe: Autocomplete/Dropdown mit bekannten Mob-/Item-/Block-Namen (kein freies Tippen, um Tippfehler-Diskussionen im Buzzer-Flow zu vermeiden).

Technischer Grundstack (Vorschlag)

- **Frontend:** HTML/CSS/JavaScript (bereits Erfahrung vorhanden)
- **Echtzeit-Backend:** Node.js + WebSockets (Socket.io) oder Firebase Realtime Database für Buzzer-Logik und Räume
- **Datenbank:** Fakten als JSON/Firestore-Collection, kategorisiert
- **Hosting:** GitHub Pages / Vercel (Frontend), kleiner Node-Server oder Firebase Functions (Backend)

Offene nächste Schritte

1. Technisches Grundgerüst (WebSocket-Server + Raum-/Buzzer-Logik) aufsetzen
2. Erste Fakten-Liste (20-30 Fragen, beide Typen) zusammenstellen
3. Solo-Modus als MVP fertigstellen, danach Multiplayer/Buzzer ergänzen